

Traffic Engineering of Management Flows by Link Augmentations on Confluent Trees

**Randeep Bhatia · Nicole Immorlica ·
Tracy Kimbrel · Vahab S. Mirrokni ·
Joseph (Seffi) Naor · Baruch Schieber**

Published online: 4 July 2007
© Springer Science+Business Media, LLC 2007

Abstract Service providers rely on the management systems housed in their Network Operations Centers (NOCs) to remotely operate, monitor and provision their data networks. Lately there has been a tremendous increase in management traffic due to the growing complexity and size of the data networks and the services provisioned on them.

Traffic engineering for management flows is essential for the smooth functioning of these networks to avoid congestion, which can result in loss of critical data such as billing records, network alarms, etc. As is the case with most intra-domain routing protocols, the management flows in many of these networks are routed on shortest paths connecting the NOC with the service provider's POPs (points of presence).

R. Bhatia (✉)

Bell Labs, Lucent Technologies, 2A-244, 600-700 Mountain Avenue, Murray Hill, NJ 07974, USA
e-mail: randeep@research.bell-labs.com

N. Immorlica · V.S. Mirrokni

MIT, 232 Vassar Street, Cambridge, MA 02139, USA

N. Immorlica

e-mail: nickle@theory.lcs.mit.edu

V.S. Mirrokni

e-mail: mirrokni@theory.lcs.mit.edu

T. Kimbrel · B. Schieber

IBM T.J. Watson Research Center, P.O. Box 218, Yorktown Heights, NY 10598, USA

T. Kimbrel

e-mail: kimbrel@us.ibm.com

B. Schieber

e-mail: sbar@us.ibm.com

J.S. Naor

Computer Science Department, Technion, Haifa 32000, Israel
e-mail: naor@technion.ac.il

This collection of paths thus forms a “confluent” tree rooted at the gateway router connected to the NOC. The links close to the gateway router may form a bottleneck in this tree resulting in congestion. Typically this congestion is alleviated by adding layer two tunnels (virtual links) that offload the traffic from some links of this tree by routing it directly to the gateway router. The traffic engineering problem is then to minimize the number of virtual links needed for alleviating congestion.

In this paper we formulate a traffic engineering problem motivated by the above mentioned applications. We show that the general versions of this problem are hard to solve. However, for some simpler cases in which the underlying network is a tree, we design efficient algorithms. In particular, we design fully polynomial-time approximate schemes (FPTAS) for different variants of this problem on trees. We use these algorithms as the basis for designing efficient heuristics for alleviating congestion in general (non-tree) service provider network topologies.

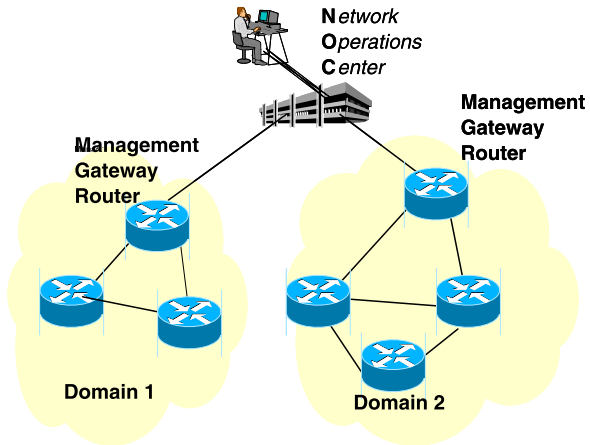
Keywords Combinatorial optimization · Dynamic programming · Traffic engineering · Confluent flows · Network management · Management flows

1 Introduction

Service providers rely on their management systems housed in their Network Operations Centers (NOCs) to remotely operate, monitor, and provision their data networks. These management systems and the network used for carrying management traffic are critical resources, necessary for provisioning customer services quickly, collecting billing data, performing software upgrades and backups, and identifying and fixing faults. The growing complexity and size of data networks, and the services provisioned on them, has resulted in a tremendous increase in management traffic. This traffic is either routed in-band with the data traffic on a common network or it is routed out-of-band on a secure, separate network, dedicated to carrying management traffic. Historically, traffic engineering and capacity planning have been done without regard to management traffic requirements. However, traffic engineering for management traffic with the goal of avoiding congestion, which results in packet losses and retransmissions, is now becoming essential for the smooth functioning of service provider networks.

Generally speaking, a service provider’s network consists of a number of management domains defined by a partition of the network topology. The domains are managed by a Network Operations Center (NOC), which is connected to its managed domains via a management gateway router (MGR) within each domain. The MGR within a domain receives and forwards management traffic from/to the routers within the domain. Typically, the MGR does not originate or carry data packets, and it is only a source or destination for management traffic. Figure 1 depicts these entities of the management domains. In connection-oriented networks such as ATM and MPLS, management and data traffic flows over connections such as Virtual Circuits (VC) and Label Switch Paths (LSP). The connections’ paths are computed using a shortest path algorithm such as Constrained Shortest Path First (CSPF). Resources such as bandwidth are reserved on the links along the path. However, unlike connections for

Fig. 1 NOC controlled management domains



data traffic which may be provided strict QoS guarantees, very little or no bandwidth or other resources are allocated for management connections and no traffic policing or shaping are performed. Thus, typically these connections are routed over shortest paths, regardless of the resource limitations of the links on the shortest paths, and irrespective of the actual amount of management traffic flowing over them. No QoS guarantees are provided to these connections and flow control is done by dropping packets at intermediate routers or switches. The management traffic flows are thus prone to congestion and losses which have an adverse impact on the normal operations of service providers' networks.

Typically, hop-by-hop routing (as is the case with most inter-domain and intra-domain routing protocols, e.g. OSPF, ISIS, BGP, etc.) is used for routing management traffic. In other words, packets are routed based only on their destination address on a shortest path to the destination. (More precisely, routers may load balance on equal cost shortest paths as in OSPF. However, we make the simplifying assumption that packets are routed on a single path to the destination.)

The collection of paths for the management connections in a management domain form a shortest path rooted tree (SPRT), rooted at the management gateway router of the domain. We refer to this tree as a "confluent" tree [5, 6]. Ideally, the management traffic load on a link in this SPRT does not exceed a certain percentage of the link's bandwidth; otherwise congestion is likely to occur. Even if a mix of data and management traffic is routed on the link, the management traffic is the first to be dropped since it has lower priority (QoS) than a customer's data traffic. By the very nature of management traffic, the links closer to the root in the SPRT carry more load and are more prone to congestion. A common and natural way to alleviate the congestion is then to create Layer 2 tunnels between a node v deep in the tree and the root. These Layer 2 tunnels are typically created as bandwidth-guaranteed connections over a separate part of the network, often using explicit routing to prevent the tunnel from taking resources away from the already congested links in the SPRT. Such a tunnel can be used to route all the data coming into a node v directly to the root, thus alleviating the congestion on all the upstream nodes on the path from v to the root in the SPRT. These Layer 2 tunnels can be thought of as virtual links between nodes of the

SPRT and the root, and are treated as any other physical link for the purpose of route computation. Once added, these links are assigned weights and affect the SPRT of the new network. By choosing low weights for these virtual links, and by changing weights on some of the links in the SPRT of the original network, the new SPRT can be made to include all the new links and to eliminate a given set of previously congested links in the SPRT. Typically, in these connection-oriented networks, paths for connections are constantly re-balanced so that they eventually settle onto the new SPRT (unless the connections are explicitly routed). Thus, in these networks the goal of traffic engineering is to determine the minimal set of Layer 2 tunnels that can be used to alleviate congestion for management flows.

The goal of traffic engineering is to enhance the performance of the network, while expending network resources economically. An efficient scheme for alleviating congestion for management traffic by virtual link augmentation must carefully balance the resulting increase in the node adjacencies and the bandwidth resources dedicated for management traffic, with the eventual goal of enhanced performance of the service provider's management systems. This is the problem we study in this paper.

The traffic engineering problem defined above is very hard to solve in its full generality (in general network topologies). Even the problem of determining whether the existing network can have a congestion-free SPRT for any link weight modifications is NP-hard [5]. We show that this remains the case even if the underlying network is a tree and new virtual links (each with its own specified capacity) may be added between any pair of nodes of the network. However, when the underlying network is a tree, and new links can only be added between the MGR and the other routers, we design an efficient algorithm for the problem based on dynamic programming (DP). Our simulation of this algorithm shows that in most cases congestion can be eliminated by adding very few links at low bandwidth. Motivated by these results, for general network topologies we propose a heuristic that runs our DP algorithm on the SPRT of the network and uses the computed virtual links for lowering congestion in the original network. Although the augmentation of the network with these new links is not guaranteed to alleviate all the congestion, our simulation results show that it indeed lowers the congestion considerably.

Our dynamic programming (DP) algorithms for tree based networks are designed to support many natural constraints such as the bandwidth and cost of the potential new links. Link costs are used to model service provider priorities, monetary costs, etc. In addition, budget constraints can be used to trade off the number of new links added against the traffic engineering gains. We are able to show that all these algorithms have very good worst case performance as well.

Tree-based networks arise naturally in other contexts including Content Distribution Networks (CDNs). For many applications, ranging from distributing rich-media content to collecting billing data, a CDN often organizes its deployment of servers in the form of a tree, rooted at the NOC, with each node forwarding data from its children to its parent and vice versa [5]. Our techniques are equally applicable to alleviate congestion in these networks.

1.1 Related Work

To the best of our knowledge the problem formulation and solutions presented in this paper are unique in the work done in this area of congestion control for management traffic. The work that comes closest to ours is that of Li et al. [16]. They study a problem of selecting the minimum number of nodes to be used for monitoring in a management domain, such that when management traffic flows on pre-determined routes from the monitored nodes to the monitoring nodes, the links stay congestion-free. They present an Integer LP formulation and present heuristics without any worst case guarantees for the problem. Their work differs from ours since we do not know the routes a priori, and thus we reduce congestion by network augmentations, and also since they assume a distributed monitoring system. Jamin et al. [13] consider a similar problem of placement of measurement instrumentation but with additional distance constraints between the monitoring nodes and their monitored nodes. They use graph-theoretic results to design heuristics for their problems. Both of these works allow for a distributed monitoring setting, where management traffic can flow to a number of monitoring agents in a given domain. However, the more commonly implemented monitoring schemes in service provider networks [2, 4] depend on a single point in the network (the MGR connected to the NOC) for actively gathering management information of a given domain. This is done for simplicity and cost effectiveness, since requiring the distribution of specialized instrumentation software and/or hardware can be cumbersome and expensive to deploy and manage inside the production network. Breitbart et al. in [4] consider such networks with a single point for gathering management information of a given domain and present the problems of finding the minimum number of nodes where measurement of either link bandwidths or link latencies is sufficient to get a network-wide view. They also extend their work to the case in which links may suffer failures.

There is a large body of work on traffic engineering of networks for data traffic. We briefly touch on this line of work in this section. The work of [8–10] deals with intra-domain traffic engineering applicable to interior gateway protocols such as OSPF and IS-IS etc. These papers show how routing can be improved by adjusting link weights based on a network-wide view of the data traffic and the topology within a routing domain. A method to alleviate link load in an IP backbone using deflection routing is proposed in [12]. The work of [1, 3] considers online routing schemes that achieve nearly optimal utilization on ISP networks even with a fairly limited knowledge of traffic demands.

Problems related to congestion for confluent flows have been considered before in the literature. In [17] these problems are studied in the context of IP routing. In this context Lorenz et al. compare traditional source-invariant IP routing with routing that considers both source and destination. They design source-invariant routing schemes with better performance guarantees. Also, in [18] this problem is considered for the purpose of traffic engineering for quality-of-service routing. Confluent flows are also studied in [15] for the purpose of minimizing the total cost of installed capacities on the links of the network under the *hose* model.

Recently, [5] and [6] studied the relation between confluent flow and the well-studied general splittable flow and unsplittable flow problems [7, 14]. Both papers

[5, 6] present approximation algorithms for the minimum congestion confluent flow problem and the maximum throughput confluent flow problem. In particular, [6] shows a tight $\Theta(\log n)$ -approximation for the minimum node-congestion confluent flow problem and a constant factor approximation for the maximum throughput confluent flow problem in general graphs for the special case in which the capacity of all nodes is the same. Our problem is different in that we are interested in augmenting the network to achieve a desired level of congestion and we are not restricted to uniform capacities. Network augmentation has been considered in different settings, especially in the context of connectivity augmentation. See [11] for a survey. To the best of our knowledge, our work is the first attempt to find a set of minimum cost links to augment a confluent flow with a guaranteed approximation factor.

1.2 Our Results

In Sect. 2 we present hardness results: the inapproximability of the problem for general network topologies and for the tree topology when augmenting links are allowed between any pair of routers. We complement these results with approximation schemes (FPTASs) for the problem in which the underlying network is a rooted tree and augmenting links can only connect to the root. Specifically, in Sect. 3 we design a dynamic programming based FPTAS for the problem of minimizing the total cost of the augmenting links needed to transform a given tree into a congestion-free tree. In case the costs of all the augmenting links are bounded by a polynomial in the size of the input, and in particular in case all these costs are identical (in which case the number of the augmenting links used is minimized), our algorithm finds an optimal solution in polynomial time. In Sect. 4 we allow for a budget constraint used to trade off the number of new links added against the traffic engineering gains. For this budget-constrained problem we also design a dynamic programming based FPTAS. In Sect. 5 we design a heuristic for our problem that is applicable to general network topologies and compare it to a greedy heuristic. Our simulations of this heuristic for general service provider topologies show that it works well in practice.

2 Preliminaries and Hardness Results

We model our network as a graph $G = (V, E)$. Each vertex $v \in V$ represents a router with s_v units of management data that must be routed between it and the management gateway. We say that v is a source of s_v units of flow. The management gateway is represented by a special *root* vertex $r \in V$. The edge set E represents links in the network. Each link, or edge e , has a hard capacity constraint c_e ; that is, edge e cannot carry more than c_e units of traffic in a congestion-free routing. A flow function from several sources to a single sink is said to be *confluent* if all the flow reaching a vertex leaves on the same edge. The edges carrying non-zero flow in a confluent flow function induce a tree in the graph.

In networks that use hop-by-hop shortest path routing, edges have weights and for each vertex v , the s_v units of flow for vertex v are routed along the shortest path from v to r . These paths form a tree rooted at vertex r , and the flows on the resulting tree are *confluent*. For the given set of weight assignments to the edges the

routed flows may violate the edge capacities. Therefore, the traffic engineering goal is to install additional edges of minimum cost together with their weight assignment so that a shortest path routing can carry the flows from all the vertices to the root. The *traffic engineering problem* is to find these additional edges and the respective weight assignment. As mentioned earlier typically it is desirable for these augmenting edges to extend from a vertex directly to the root. We call the version of the problem with this additional restriction the *restricted traffic engineering problem*. To emphasize the distinction between the two versions of the problem we sometimes call the version without this restriction the *general traffic engineering problem*. We show below that the general traffic engineering problem is inapproximable even for trees if $P \neq NP$. Our reduction is from the NP-hard *confluent flow* problem, defined as follows.

Definition 1 Given a graph $G = (V, E)$ with root $r \in V$, capacities c_e on the edges, and in which each vertex v is a source of s_v units of flow, decide whether all the sources can be routed to the root confluent (i.e., the edges with non-zero flow form a tree).

Theorem 1 *There is no approximation algorithm for the general traffic engineering problem for trees unless $P = NP$.*

Proof Suppose we are given an instance of the confluent flow problem on a (connected) graph $G = (V, E)$, and let $T = (V, E')$ be a spanning tree of G . Consider an instance of the general traffic engineering problem for tree T where the cost of adding any edge in $E \setminus E'$ is zero, and any other possible external edges have cost infinity. Then G has a confluent flow if and only if there is a set of augmenting edges A of cost zero and an assignment of weights to $E' \cup A$ such that the routing is feasible. Thus, an approximation algorithm for the general traffic engineering problem for T would yield a solution to the confluent flow problem instance. $\square$

In the rest of the paper we consider only the restricted traffic engineering problem defined formally as follows.

Definition 2 The inputs to the restricted traffic engineering problem are a graph $G = (V, E)$ with root $r \in V$, and capacities c_e together with weights $w_e > 0$ on the edges. Each vertex v is a source of s_v units of flow. For each vertex v , c_v units of flow can be routed directly from v to the root r incurring cost p_v . The *restricted traffic engineering problem* is to find a set of edges A of minimum cost from vertices directly to the root, and a weight assignment for edges $E \cup A$, such that the routing on the shortest paths tree is feasible.

A proof similar to that of Theorem 1 shows that the restricted traffic engineering problem is inapproximable for general graphs. Therefore, we further restrict ourselves and study the restricted traffic engineering problem only for trees. Applying our tree based algorithms to the shortest paths tree of a given graph, we obtain a heuristic (with no bounded performance guarantees) for the restricted traffic engineering problem for general graphs.

When we restrict our algorithms to trees, we have the advantage that we may not need to change the weights of the edges belonging to the original network. Consider the following procedure for choosing weights for the edges in the augmentation set. We can proceed sequentially as follows. At each step we add an edge from a vertex to the root and delete an edge from the original tree. Suppose that edge (v, r) is added and edge (x, y) is deleted, where the weight of the (unique) path from v to x is W_1 and the weight of the (unique) path from y to r is W_2 . Then, the weight of edge (v, r) should satisfy:

$$\text{For vertex } x: \quad W_1 + w(v, r) < w(x, y) + W_2,$$

$$\text{For vertex } y: \quad W_2 < w(x, y) + W_1 + w(v, r).$$

That is,

$$W_2 - W_1 - w(x, y) < w(v, r) < W_2 - W_1 + w(x, y),$$

and there is a feasible choice for $w(v, r)$. Note that if all the edge weights computed by this procedure are non-negative, then with this choice of weight setting for the augmenting edges we do not need to change any edge weights in the original network. Even when some weights turn out to be negative, we can use the standard edge weight modification procedure to make all weights non-negative, since it can be shown that all cycles in the graph remain non-negative by our choice for $w(v, r)$. However, this would also require changing some edge weights in the original graph. For the above reasons, from now on until Sect. 5 we ignore the assignment of edge weights, and thus our problem becomes a *network augmentation problem* defined as follows.

Definition 3 The input to the network augmentation problem is a graph $G = (V, E)$ with root $r \in V$ and capacities c_e on the edges. Each vertex v is a source of s_v units of flow. For each vertex v , c_v units of flow can be routed directly from v to the root r for price p_v . The *network augmentation problem* is to find a set of edges of minimum cost from vertices (directly) to the root, and a confluent flow satisfying all the sources s_v .

We also define a natural variation of this problem which we call the *budget-constrained network augmentation problem*. Here the service provider has a budget B and the goal is to find a set of edges of total cost at most B , joining the vertices directly to the root, such that by using these edges the maximum amount of management traffic can be routed with no congestion on the augmented graph. Note that here we allow a source to send a fractional amount of its s_v units of traffic.

We prove that both of our augmentation problems for trees are at least as hard as the weakly NP-hard knapsack problem. In the knapsack problem, we are given a finite capacity knapsack and a set of items, where each item has a weight and a value. The goal is to find a maximum value subset of the items such that its weight does not exceed the capacity of the knapsack.

Theorem 2 *The network augmentation problem is weakly NP-hard even for trees.*

Proof Given an instance of the knapsack problem, let C denote the capacity of the knapsack, and suppose we are given n items, where item v has weight s_v and value p_v . Construct a tree T with n leaves, where leaf v is a source of s_v units of flow. These leaves all stem from a single vertex u which is a source of 0 units of flow. The edge from a leaf v to vertex u has capacity $c_{(v,u)} = s_v$. Vertex u is adjacent to the root r through an edge of capacity C . We can buy an edge from any leaf v to the root at cost p_v and this edge has capacity $c_v = s_v$. We prevent the solution from buying an edge from u to the root by assigning this edge infinite cost and zero capacity. It is not hard to see that the optimal solution of this instance of the network augmentation problem yields an optimal solution to the knapsack instance. $\square$

Theorem 3 *The budget-constrained network augmentation problem is weakly NP-hard.*

Proof Given an instance of the knapsack problem with capacity C and n items of weights p_v and values s_v , construct a tree T with zero-capacity edges and n leaves, where leaf v is a source of s_v units of flow. For each vertex v , we can add an edge of cost p_v with capacity s_v from v to the root. The budget constraint B in the network augmentation problem is equal to the knapsack size C . It is easy to see that an optimal solution of the knapsack instance can be obtained from an optimal solution of the budget-constrained network augmentation instance. $\square$

Because of the NP-hardness of the problems, we consider approximation algorithms for our augmentation problems. The best approximation we can hope for is a *fully polynomial-time approximation scheme* (FPTAS). An approximation algorithm is an FPTAS for a minimization (maximization) problem if, for any $\epsilon > 0$, it finds a $(1 + \epsilon)$ -approximation ($(1 - \epsilon)$ -approximation) solution in time which is polynomial in the input size and $1/\epsilon$. We present an FPTAS for several versions of our problem, and optimal algorithms for some restricted instances.

In the rest of this paper, we consider the network augmentation problem only for trees. Thus, we are given an initial tree T which we must augment and change into a new tree T' on which flow is routed. For ease of discourse, we consider vertices and edges to be oriented with respect to the original tree T . We say that the root of the input tree T is at the *top*, and that a vertex v (edge e) is *below* vertex v' (edge e') if it is farther away from the root than v' (e'). For an edge (x, y) in the tree we assume that vertex x is below vertex y . For a vertex v , we denote by T_v the subtree rooted at v . We will write $V(G)$ to indicate the vertex set of graph G and $E(G)$ to indicate the edge set of graph G .

3 Routing Flows at Minimum Cost

In this section we present a dynamic-programming-based FPTAS for the network augmentation problem on trees. We describe the dynamic program for the case of a binary tree only. The general case can be reduced to this case with only a constant factor increase in the size of the table maintained in the dynamic program and with

only a constant factor increase in the running time. For simplicity we omit the details of this construction. (We note that the general case cannot be handled by simply adding dummy nodes to make the tree binary, since a confluent flow in the resulting binary tree may not necessarily translate to a confluent flow in the original tree.)

For each edge $e = (v, u) \in E(T)$ (with u above v) and each possible cost ρ , we find the smallest possible flow $f(e, \rho)$ on edge e while spending at most ρ in the subtree T_v rooted at v . That is, we want to satisfy the maximum possible flow from sources in T_v , including v , and route as little flow as possible through e , or even reverse the flow on e and carry flow from sources outside T_v through e into T_v and eventually to the root through purchased edges.

Note that in our setting, a flow f is a function from a subset of edges E' (exactly those edges on which flow is routed) to the real numbers which describes how flow is routed on edges E' . A positive flow of $f(e, \rho)$ on edge e indicates that there are $f(e, \rho)$ units of flow from sources in T_v unsatisfied by flow f . We need to route these $f(e, \rho)$ units to the root through u , so edge e will carry flow $f(e, \rho)$ toward the root. A negative flow of magnitude $|f(e, \rho)|$ on e indicates that all the flow from sources in T_v is satisfied by purchased edges to the root from vertices below e , and we can push additional flow through u into T_v while maintaining feasibility of the solution. Edge e carries up to $|f(e, \rho)|$ units of flow in the downward direction, from u to v .

Consider an edge $e = (v, u) \in E(T)$ and a cost ρ . Suppose that x and y are the two children of v . We show how to compute $f((v, u), \rho)$ given the values of $f((x, v), \rho')$ and $f((y, v), \rho')$ for all $\rho' \leq \rho$. The value of $f((v, u), \rho)$ is given as the minimum possible flow obtained by spending at most ρ in the subtree T_v . This budget can be spent to purchase edges in the subtrees T_x and T_y , and to purchase the edge from v to the root. Let ρ_x and ρ_y be the amounts spent to purchase edges in the subtrees T_x and T_y , respectively. Let Z_v denote a binary decision variable indicating whether the edge from v to r is purchased. Clearly, in order not to exceed the budget we must have

$$1. \quad \rho \geq \rho_x + \rho_y + Z_v \cdot p_v.$$

Let $Z_{(x,v)}$ and $Z_{(y,v)}$ denote binary decision variables indicating whether flow is carried on edges (x, v) and (y, v) , respectively, in either direction. Clearly, if $f((x, v), \rho_x) > 0$ then $Z_{(x,v)}$ must be 1 since the flow $f((x, v), \rho_x)$ needs to be carried on (x, v) . The same applies also to y and thus we must have

$$2. \quad \text{If } f((x, v), \rho_x) > 0 \text{ (resp. } f((y, v), \rho_y) > 0) \text{ then } Z_{(x,v)} = 1 \text{ (resp. } Z_{(y,v)} = 1).$$

Note that edge (x, v) can be used to carry flow from v to x , in which case $f((x, v), \rho_x)$ must be negative. The same applies also to y . Since the flow needs to be confluent only one edge among the four edges (x, v) , (y, v) , (v, u) and (v, r) can be used to carry flow from v . Thus

$$3. \quad \text{At most one of } f((v, u), \rho), -Z_{(x,v)} \cdot f((x, v), \rho_x), -Z_{(y,v)} \cdot f((y, v), \rho_y), \text{ and } Z_v \text{ is positive.}$$

To compute $f((v, u), \rho)$ we first compute

$$g((v, u), \rho) = \min\{s_v - Z_v \cdot c_v + Z_{(x,v)} \cdot f((x, v), \rho_x) + Z_{(y,v)} \cdot f((y, v), \rho_y),$$

where the minimum is over the reals ρ_x, ρ_y , and the binaries $Z_{(x,v)}, Z_{(y,v)}, Z_v$ that satisfy the constraints 1–3 above. If $|g((v, u), \rho)| \leq c_{(v,u)}$ then this flow is feasible

and we set $f((v, u), \rho) = g((v, u), \rho)$. If $g((v, u), \rho) < -c_{(v,u)}$, then this flow is infeasible and we set $f((v, u), \rho) = -c_{(v,u)}$. Finally, if $g((v, u), \rho) > c_{(v,u)}$, then there is no solution of cost ρ , since we need to carry more than $c_{(v,u)}$ units of flow on the edge (v, u) , and thus we set $f((v, u), \rho) = \infty$. In case there is no solution for a cost ρ the flow value of infinity will propagate to an edge incident on the root r . If $f((v, r), \rho) = \infty$ for a child v of r and all values of ρ , there is no feasible solution at any cost.

Note that as a degenerate case, it is possible that if $s_v = 0$ (and only in this case), then all of the flows out of v are non-positive (a negative would indicate capacity that is available but unused) and v is an isolated vertex in the final solution.

First, we show how to solve the problem optimally when the costs p_v are polynomially bounded and have polynomially many distinct values (i.e., $p_v \in [0..n^c]$ for some constant c) and the tree is binary. Note the maximum total cost is n^{c+1} . We build a dynamic programming table that indicates the amount of flow that must be sent on edge (v, u) given that ρ is spent in the subtree rooted at v , for each cost $\rho \leq n^{c+1}$ for all $e \in E$.

For each edge $e = (v, u) \in T$, we compute the minimum flow from v to u , or the maximum amount of flow $f(e, \rho)$ that we can feasibly push into T_v , for cost ρ . Note that if we can push flow into T_v (and eventually to the root through edges in T_v and some purchased edge), then the flow on e will be negative. If we must carry some flow from sources in T_v through e in the original direction from v to u , then the flow on e will be positive. Thus, we want to minimize $f(e, \rho)$ subject to the cost and feasibility constraints.

We begin with the leaves and compute the possible flows for each edge and for each cost in bottom-up fashion. For an edge e incident to a leaf v , if $c_e \geq s_v$, then edge e can carry flow s_v for each $\rho < p_v$. If $c_v \geq s_v$, then edge e can carry flow $-\min(c_e, (c_v - s_v))$ for each $\rho \geq p_v$.

Now we would like to compute the set of possible flows for each cost on an edge (v, u) given the costs and corresponding flows on the edges (x, v) and (y, v) immediately below (v, u) in T . Let ρ_x and ρ_y be a pair of costs from the tables for (x, v) and (y, v) respectively. Let $f((x, v), \rho_x)$ and $f((y, v), \rho_y)$ be the corresponding flows on (x, v) and (y, v) .

There are three cases to consider:

- If $f((x, v), \rho_x) > 0$ and $f((y, v), \rho_y) > 0$, then, if we can do so without violating capacities,
 - We can route all leftover flow below e up through e at no additional cost: $\rho = \rho_x + \rho_y$, $f(e, \rho) = f((x, v), \rho_x) + f((y, v), \rho_y) + s_v$.
 - We can route all leftover flow below e and possibly some flow from sources above e on edge (v, r) at additional cost p_v : $\rho = \rho_x + \rho_y + p_v$, $f(e, \rho) = -\min(c_e, c_v - (f((x, v), \rho_x) + f((y, v), \rho_y) + s_v))$.
- If $f((x, v), \rho_x) > 0$ and $f((y, v), \rho_y) \leq 0$ (the case $f((x, v), \rho_x) \leq 0$ and $f((y, v), \rho_y) > 0$ is analogous), then, if we can do so without violating capacities,
 - We can route all leftover flow below e and possibly some flow from sources above e on edge (y, v) at no additional cost: $\rho = \rho_x + \rho_y$, $f(e, \rho) = -\min(c_e, |f((y, v), \rho_y)| - (f((x, v), \rho_x) + s_v))$.

- We can route all leftover flow below (x, v) , s_v , and possibly some flow from sources above e on edge (v, r) at additional cost p_v (in this case, edge (y, v) will have zero flow): $\rho = \rho_x + \rho_y + p_v$, $f(e, \rho) = -\min(c_e, c_v - (f((x, v), \rho_x) + s_v))$.
- We can route all leftover flow below (x, v) and s_v up edge e at no additional cost (in this case, edge (y, v) will have zero flow): $\rho = \rho_x + \rho_y$, $f(e, \rho) = f((x, v), \rho_x) + s_v$.
- If $f((x, v), \rho_x) \leq 0$ and $f((y, v), \rho_y) \leq 0$, then, if we can do so without violating capacities,
 - We can route flow s_v and possibly some flow from sources above e on edge (x, v) at no additional cost (in this case, edge (y, v) will have zero flow):

$$\rho = \rho_x + \rho_y, f(e, \rho) = -\min(c_e, |f((x, v), \rho_x)| - s_v).$$

- We can route flow s_v and possibly some flow from sources above e on edge (y, v) at no additional cost (in this case, edge (x, v) will have zero flow):

$$\rho = \rho_x + \rho_y, f(e, \rho) = -\min(c_e, |f((y, v), \rho_y)| - s_v).$$

- We can route flow s_v on edge (v, r) at additional cost p_v (in this case, edges (x, v) and (y, v) will have zero flow): $\rho = \rho_x + \rho_y + p_v$ and $f(e, \rho) = -\min(c_e, c_v - s_v)$.
- We can route flow s_v on edge e at no additional cost (in this case, edges (x, v) and (y, v) will have zero flow): $\rho = \rho_x + \rho_y$, $f(e, \rho) = s_v$.

After performing all such computations, for each resulting cost ρ , we add an entry to our table F_e . The value of the entry is the flow f achieved at cost ρ for which $f(e, \rho)$ is minimized.

For the final step, suppose x and y are children of the root r . The algorithm reports the value $\rho_x + \rho_y$ where ρ_x and ρ_y are the minimum costs for which $|f((x, r), \rho_x)| \leq c_{(x,r)}$ and $|f((y, r), \rho_y)| \leq c_{(y,r)}$, respectively.

The standard mechanism of recording pointers between entries in the dynamic programming tables and backtracking through them yields an algorithm to find an optimal flow as well as its cost.

Theorem 4 *The above procedure finds an optimal solution in polynomial time if all edge costs are integers bounded by a polynomial.*

Proof Suppose that for all v , edge cost p_v is an integer in the range $[0..n^c]$, for some constant c . We show that in this case the running time is polynomial in n . Note that there are only n^{c+1} distinct values for the costs of flows on an edge and all edges below it. Thus we must find for each vertex a table of at most n^{c+1} values. Each of these is found by considering at most n^{c+1} combinations of flows on the edges below it as specified by the recurrence relation.

By construction, every flow considered for each edge is constructed from feasible flows for the subtrees below it in such a way that the resulting flow is confluent and does not violate capacities. Furthermore, it routes all flow from sources in the subtree

below the edge either to the root through purchased edges below it, up through the edge in question, or by purchasing a new edge, to eventually reach the root. Therefore, every flow considered in the final step of the algorithm is a feasible flow.

It remains to show that the flow computed is optimal. Fix an optimal solution. Let ρ_e^* be the cost paid by the optimal solution in the subtree below e , and let f_e^* be the flow on e . (Recall that f_e^* may be negative.) We claim that the value of $f(e, \rho_e^*)$ computed for the flow on edge e for cost ρ_e^* is at most f_e^* . We prove the claim by induction.

For an edge $e = (v, u)$ immediately above a leaf v , suppose the optimal solution does not buy the edge (v, r) from v to the root. The algorithm computes $f(e, 0) = s_v$ as needed. Suppose the optimal solution buys (v, r) incurring cost p_v . Then it routes at most c_v through this edge and the flow on e is at least $\max(s_v - c_v, -c_e)$. (Again, this may be negative.) This is the value $f(e, p_v)$ specified in the recurrence and computed by the algorithm.

Consider an edge $e = (v, u)$ with edges $e_x = (x, v)$ and $e_y = (y, v)$ immediately below e . Let T_v , T_x , and T_y be the subtrees rooted at v , x , and y respectively, let ρ_e^* , $\rho_{e_x}^*$, and $\rho_{e_y}^*$ denote the costs paid by the optimal solution in these subtrees, and let f_e^* , $f_{(x,v)}^*$, and $f_{(y,v)}^*$ denote the flows in the optimal solution. Let $Z_{(x,v)}^*$, and $Z_{(y,v)}^*$, indicate whether the optimal solution places flow on edges (x, v) and (y, v) , and let Z_v^* indicate whether the optimal solution purchases the edge from v to r . By the induction hypothesis, $f((x, v), \rho_x^*) \leq f_{(x,v)}^*$ and $f((y, v), \rho_y^*) \leq f_{(y,v)}^*$. Thus the value for the flow on e at cost ρ^* computed using these settings of the decision variables Z_v , $Z_{(x,v)}$, and $Z_{(y,v)}$ is at most f_e^* , and the minimum $f(e, \rho_e^*)$ specified in the recurrence and computed by the algorithm is at most this value as well. $\square$

Consider a uniform instance, in which all costs are the same (and thus the objective is to minimize the number of augmentations). Note that the above procedures finds an optimal solution in polynomial time for such an instance.

Consider an arbitrary instance. We can round the prices of that instance and use the dynamic program to find an approximately optimal solution. The rounding works by guessing the maximum price of an edge bought in an optimal solution, which is a lower bound on the overall cost of an optimal solution.

Algorithm 1 (Augment)

Input:

An instance of the network augmentation problem.

Positive $0 < \epsilon \leq 1$. (We compute a $(1 + \epsilon)$ -approximation for the problem.)

If the given instance is feasible as is, terminate.

For each $P \in \{p_v\}$, let $S_P = \{v \mid p_v \leq P\}$.

Let $K_P = \frac{\epsilon P}{n}$.

For each $v \in S_P$, let $p_v^P = \lceil \frac{p_v}{K_P} \rceil$.

For each $v \notin S_P$, let $p_v^P = \infty$ (i.e., set the decision variable $Z_v = 0$ in the dynamic program).

Solve the instance optimally for prices p_v^P ; denote this solution by O_P .

Output the solution O_P with minimum cost according to the original (unrounded) prices.

Theorem 5 *The above algorithm is an FPTAS for the network augmentation problem with arbitrary costs.*

Proof First notice that the algorithm runs in time polynomial in n and $1/\epsilon$ as we solve n dynamic programs and the prices in each dynamic program are bounded by $\lceil n/\epsilon \rceil$.

Let $C(S)$ be the cost of solution S in the original instance and $C_P(S)$ be the cost in the rounded instance with rounding/cutoff parameter P . For any price p_u feasible given the cutoff, $K_P p_u^P - K_P \leq p_u \leq K_P p_u^P$. Therefore,

$$K_P C_P(S) - nK_P \leq C(S) \leq K_P C_P(S)$$

provided S does not use edges of cost more than P .

Let O_P be the solution output by our algorithm, P^* be the maximum price of an edge in an optimal solution, O_{P^*} be the solution considered by our algorithm for $P = P^*$ (optimal for the rounded instance with rounding/cutoff parameter P^*), and O be the optimal solution to the original instance. Then,

$$\begin{aligned} C(O_P) &\leq C(O_{P^*}) \\ &\leq K_{P^*} C_{P^*}(O_{P^*}) \\ &\leq K_{P^*} C_{P^*}(O) \\ &\leq C(O) + nK_{P^*} \\ &= C(O) + \epsilon P^* \\ &\leq (1 + \epsilon)C(O). \end{aligned}$$

□

4 Maximizing Throughput with Budget Constraint

Next we consider the budget-constrained network augmentation problem. In this problem, we can add edges of total cost no more than a given budget B , and we seek to maximize the amount of flow routed subject to this budget constraint. Again, we require that flow is routed confluent. However, we permit solutions to route only part of the flow sourced at a node. Again, the best we can hope for is to find a fully polynomial approximation scheme (recall the hardness result in Sect. 2).

We now describe a natural dynamic program for solving this problem when the source values and capacities are polynomially bounded. We use this dynamic program to design an FPTAS for the case in which we have a lower bound on the capacity of each edge in terms of the size of the maximum source. Then we generalize this FPTAS to an FPTAS for the general budget-constrained network augmentation problem. As before we describe the dynamic programming for the case of a binary tree only and note that the general case can be reduced to this case as well.

Assuming sources and capacities are polynomially bounded, a variation on the algorithm of Sect. 3 solves the problem optimally in polynomial time. As before, our algorithm constructs a table for each edge. However, the table is now indexed by the

form of the solution—the flows on the edges and flows satisfied by added edges—and the entries of the table are the costs. More specifically, for an edge e , for each possible flow on e , and for each amount of flow already satisfied by edges added below e , we store the cost of such a solution.

We begin with the leaves and compute the costs in bottom-up fashion. For an edge e incident to a leaf v , edge e can carry any flow $f_e \leq \min(s_v, c_e)$ for cost $\rho = 0$ without satisfying any flow whose source is at v through (v, r) . If $c_v \geq s_v$, then edge e can carry any flow $0 \geq f_e \geq -\min(c_e, (c_v - s_v))$ for cost $\rho = p_v$ while satisfying $\min(s_v, c_v)$ flow whose source is at v . Of course, we only need to remember the best solution for a given cost entry, and so the table for a leaf will have up to two entries: flow $\min(s_v, c_e)$ and satisfied flow 0 at cost 0; flow $-\min(c_e, \max(c_v - s_v, 0))$ and satisfied flow $\min(s_v, c_v)$ at cost p_v .

The inductive step for an edge e that is not incident to a leaf is the same as before, only now we must also record the amount of flow satisfied below e as well as the flow through e and the cost. We simply add the amounts of flow satisfied below each of the children's edges. As in the case of an edge incident to a leaf above, we must cap the flows through the given edge and the purchased edge to the root, if used, by their capacities.

For the final step, we combine the tables of the edges incident to the root as described in the inductive step. We discard all solutions whose costs are greater than the budget B , and return the remaining solution with the maximum amount of satisfied flow.

It is not hard to see that this algorithm computes the optimal solution in polynomial time when the capacities and sources are polynomially bounded. We can use this algorithm to get an FPTAS for an instance with arbitrary capacities and sources.

First, we present an FPTAS for the case in which capacities are not too small compared to sources; that is, the ratio of the maximum satisfiable flow from a single source to the minimum capacity is bounded from above. Let us introduce some definitions and notation first.

Definition 4 Let D be the largest amount of flow from a single node which can be routed to the root by spending at most B (D is a lower bound on OPT). Note that D can be just a fraction of the flow sourced at the node.

By the above definition of D , it is straightforward to see that $D \leq \text{OPT} \leq nD$ where OPT is the value of the optimal solution.

Definition 5 Let $c_{\min}$ be the minimum over all edges $e \in E$ of capacity c_e and over all possible added edges (v, r) of capacity c_v , i.e., $c_{\min} = \min(\min_{e \in E} c_e, \min_{v \in V} c_v)$. Let p be the maximum of 1 and $\lceil \log_n \frac{D}{c_{\min}} \rceil$. Note that $\frac{D}{c_e} \leq n^p$ for all $e \in E$, and $\frac{D}{c_v} \leq n^p$ for all $v \in V$.

The Algorithm BCNA1 for the budget-constrained network augmentation problem is as follows:

Algorithm 2 (BCNA1)*Input:*Instance I of the budget-constrained network augmentation problem.Positive $0 < \epsilon \leq 1$ (We will find an $(1 - \epsilon)$ -approximation for the problem.)Let $\epsilon' = \epsilon'' = \frac{\epsilon}{2}$.Cap all sources at D and capacities at nD (this does not change OPT, so from now on we will assume our instance originally satisfied this condition).Let $K = \frac{\epsilon' D}{n^p}$.For each source s_v , let $s'_v = \lceil \frac{s_v}{K} \rceil$.For each capacity c_e , let $c'_e = \lceil \frac{c_e}{K} \rceil$.For each capacity c_v , let $c'_v = \lceil \frac{c_v}{K} \rceil$.Solve the instance optimally for sources s'_v and capacities c'_e and c'_v to get solution O' .Scale down O' by a factor of $(1 - \epsilon'')$.Output the corresponding (unrounded) solution S . That is, if O' routes $\alpha s'_v$ fraction of the flow sourced at node v along edge e , then O routes $\alpha(1 - \epsilon'')s_v$ flow sourced at node v along edge e .

Theorem 6 *The Algorithm BCNA1 is an FPTAS for the budget-constrained network augmentation problem when $p = \lceil \log_n \frac{D}{c_{\min}} \rceil$ is constant.*

Proof It is easy to see that capping sources at D and capacities at nD will not change the value of the optimal solution. Now, we need to bound the number of different flows on edges in the rounded instance. In fact, after rounding, all sources are integers between 1 and $\lceil \frac{2n^p}{\epsilon} \rceil$ and capacities are integers between 1 and $\lceil \frac{2n^{p+1}}{\epsilon} \rceil$. Thus each flow on an edge is the sum of sources and capacities and is an integer between 1 and $\lceil \frac{2n^{p+2}}{\epsilon} \rceil$, which is polynomial in n^p and $\frac{1}{\epsilon}$. Since by our assumption p is constant, the running time of the dynamic program is polynomial.

Define a solution Q by the fraction of s_v , for every $v \in V$, that is routed on each original or added edge. For a solution Q in the original instance, let $F(Q)$ be the amount of flow routed by solution Q . Similarly, for a solution Q' in the rounded instance, let $F'(Q')$ be the amount of flow routed by a solution Q' . Consider a feasible solution Q' in the rounded instance. Note that Q' defines also a solution in the original instance. However, this solution may not be feasible since some of the capacity constraints may be violated. For any source s_u , $Ks'_u - K \leq s_u \leq Ks'_u$. Therefore, $KF'(Q') - nK \leq F(Q') \leq KF'(Q')$.

Let S be the solution output by our algorithm, O' be the optimal solution of the rounded instance, and O be the optimal solution of the original instance. Now, consider the budget-constrained network augmentation instance with sources Ks'_v as sources and Kc'_e and Kc'_v as capacities. The value of the optimal solution for this instance is $KF'(O')$. The sources and capacities in this instance are not smaller than the corresponding sources and capacities in the original instance, and thus the value of the optimal solution of this instance should not be smaller than that of the original instance. Therefore, $KF'(O') \geq F(O)$. It follows that

$$\begin{aligned} F(S) &= (1 - \epsilon'')F(O') \\ &\geq (1 - \epsilon'')(KF'(O') - nK) \end{aligned}$$

$$\begin{aligned}
&\geq (1 - \epsilon'')(F(O) - nK) \\
&= (1 - \epsilon'')(F(O) - \frac{\epsilon' D}{n^{p-1}}) \\
&\geq (1 - \epsilon'')(F(O) - \epsilon' D) \\
&\geq (1 - \epsilon'')(1 - \epsilon')F(O) \\
&\geq (1 - \epsilon)F(O).
\end{aligned}$$

Clearly, the cost of S is at most the budget B . It remains to prove that S is feasible. Let f be the flow on edge e in S and f' be the flow on edge e in O' . We know that $f'_e \leq c'_e$ and we need to prove that $f_e \leq c_e$. First, we prove that $(1 - \epsilon'')c'_e \leq c'_e - 1$.

$$\frac{D}{c_e} \leq n^p \Rightarrow \epsilon'' = \epsilon' \geq \frac{\epsilon' D}{n^p c_e} \Rightarrow \epsilon'' \geq \frac{1}{c'_e} \Rightarrow (1 - \epsilon'')c'_e \leq c'_e - 1.$$

Now,

$$f(e) \leq K(1 - \epsilon'')f'_e \leq K(1 - \epsilon'')c'_e \leq K(c'_e - 1) = Kc'_e - K \leq c_e.$$

Thus the flow f in solution S is feasible. $\square$

The disadvantage of Algorithm BCNA1 is that its running time depends on the ratio $\frac{D}{c_{\min}}$ for an arbitrary instance. In the following we overcome this problem by removing a set of edges with small capacities and proving that removing this set of edges does not change the value of the optimal solution. Then, using the Algorithm BCNA1 in the new instance, we design an FPTAS BCNA2 for the general budget-constrained network augmentation problem.

The Algorithm BCNA2 is as follows:

Algorithm 3 (BCNA2)

Input:

Instance I of the budget-constrained network augmentation problem.

Positive $0 < \epsilon \leq 1$ (We will find an $(1 - \epsilon)$ -approximation for the problem).

Let $\epsilon' = \epsilon'' = \frac{\epsilon}{2}$.

Consider the set E_1 of all edges of the capacity less than $\frac{D}{n^{q+1}}$ where $q = \log_n \frac{1}{\epsilon'}$. Remove all edges in E_1 from the graph. Let the resulting instance after removing these edges be I' .

Call Algorithm BCNA1 on instance I' and parameter ϵ'' . Let the output be O' .

Output O' .

Theorem 7 *The Algorithm BCNA2 is an FPTAS for the budget-constrained network augmentation problem.*

Proof In the instance I' , $\frac{D}{c_{\min}} \leq n^{q+1}$, thus from Theorem 6, the running of Algorithm BCNA1 is a polynomial in $n^{\max(1, q+1)}$ and $\frac{1}{\epsilon''}$, thus it is a polynomial in $n^{1 + \log_n \frac{1}{\epsilon'}} = \frac{n}{\epsilon'} = \frac{2n}{\epsilon}$ and $\frac{1}{\epsilon}$.

Let OPT' be the value of the optimal solution of instance I' . From Theorem 6, we know that $F(O') \geq (1 - \epsilon'')\text{OPT}'$. Let OPT be the value of the optimal solution of instance I . Since each removed edge can carry at most $\frac{D}{n^{q+1}}$ flow and there are at most n separate edges (or paths) to the root, $\text{OPT} \leq \text{OPT}' + \frac{D}{n^q}$. Therefore,

$$\text{OPT}' \geq \text{OPT} - \frac{D}{n^q} \geq \text{OPT} \left(1 - \frac{1}{n^q}\right).$$

By the definition of q , $n^q = \frac{1}{\epsilon'}$, thus, $\frac{1}{n^q} = \epsilon'$, thus $\text{OPT}' \geq \text{OPT}(1 - \epsilon')$. From this inequality, we have

$$\begin{aligned} F(O') &\geq (1 - \epsilon'')\text{OPT}' \\ &\geq (1 - \epsilon'')(1 - \epsilon')\text{OPT} \\ &\geq (1 - \epsilon)\text{OPT} \end{aligned}$$

as desired. This completes the proof of the FPTAS. $\square$

5 Heuristics for General Network Topologies and Simulations

In this section we present our heuristic for alleviating congestion in more complex non-tree service provider network topologies. The heuristic uses an algorithm for the network augmentation problem for trees as a subroutine. We use simulation to demonstrate the performance of the heuristic using the dynamic programming (DP) algorithm presented in Sect. 3.

For comparison, we benchmark the heuristics also using a greedy algorithm for the network augmentation problem for trees, described below, which is a natural alternative to the DP algorithm.

5.1 Greedy Algorithm for Tree Topologies

Informally, the greedy algorithm looks for those nodes “underneath” a congested edge, whose augmenting edge can bypass some of the traffic on the congested edge directly off to the root. The greedy algorithm repeatedly picks that congested edge and a node underneath it which can be used to bypass the most traffic. The algorithm makes this choice with no regard to the cost or the capacity of the augmenting edges used for the bypass. The algorithm repeats until there are no more congested edges or there are no bypass nodes under congested edges. Note that the greedy algorithm can be efficiently implemented and it is a natural alternative for alleviating congestion in tree topologies.

More formally, the algorithm works as follows. We say an edge is *tight* if the total flow of the nodes that route traffic up to the root through this edge exceeds the edge capacity. An edge e is *maximally tight* if it is the lowest tight edge in the given tree. From this definition, it is clear that if there are no maximally tight edges, then the given network is congestion-free. We say a vertex is a *good* vertex if its augmenting edge has not been used yet and the total flow of the nodes that route traffic up to the

root through this node is at most the capacity of the augmenting edge that connects this node to the root. For any maximally tight edge e , we call vertex v a *maximal good vertex for edge e* if v is the uppermost good vertex below e .

Given tree T with root r , the greedy algorithm finds a maximally tight edge e . It then finds the maximal good vertex v for e such that the flow through v is the largest among all maximal good vertices for e . The algorithm adds edge (v, r) and removes the edge just above v in the original tree, and recurses on the remaining tree until there are no more maximally tight edges, or until there are no more good vertices for some maximally tight edge. In the latter case no solution is found.

Algorithm 4 (Greedy)

If there is no maximally tight edge, terminate successfully. If a tight edge has no good vertices, then terminate unsuccessfully.

Otherwise, let e be a maximally tight edge. Find all maximal good vertices for edge e , say $S = \{v_i\}$.

Find the vertex $v_i \in S$ such that the flow from v to its parent is largest.

Add edge (v_i, r) and remove the edge just above v_i to form tree T' .

Repeat the algorithm for tree T' .

5.2 Heuristics for General Network Topologies

The heuristic works by computing a shortest path rooted tree (SPRT) of the given network. It then computes the optimal congestion-free augmentation tree T of this SPRT using the DP presented in Sect. 3. An alternative heuristic that serves as a benchmark for comparison purposes uses the Greedy algorithm presented in Sect. 5.1 instead of the DP for computing the augmentation tree T of the SPRT. These heuristics attempt to force the management flows onto the paths in T by setting the costs of the new links to 0 and by setting the costs of the links that are not in T but were in the original SPRT to a large value. Note that as implied by our hardness results these heuristics are not guaranteed to eliminate all the congestion. However our simulation results that follow below show that the DP based heuristic is effective for reducing congestion in service provider networks. The pseudo-code for the heuristic is presented below:

Algorithm 5 (General)

Compute the shortest path rooted tree (SPRT) T (based on the provisioned link weights) rooted at the management gateway router (MGR).

Use the tree algorithm (DP in Sect. 3 or Greedy in Sect. 5.1) to modify the SPRT, using new links to obtain a congestion-free rooted tree T' .

Augment G with the links in $T' - T$ and set the weights of these links to 0, so they are likely to be in the new SPRT.

Set the weights of the links in $T - T'$ to a large value so that they are no longer in the new SPRT.

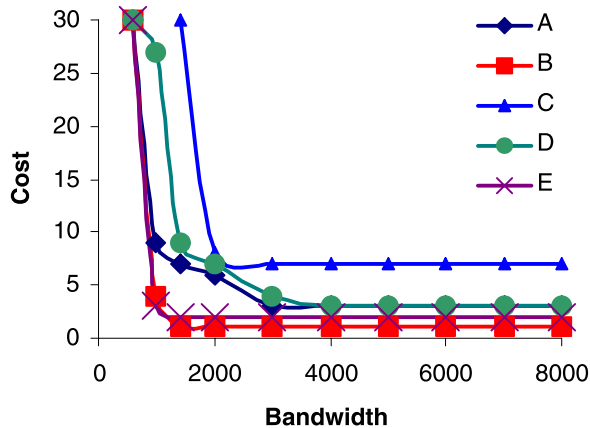
5.3 Simulations

Next we present simulation results for the heuristic for the basic network augmentation problem with general network topologies as defined by Algorithm 5. Our main goal is to evaluate the performance of this heuristic for alleviating congestion of management traffic in general network topologies. In addition we are interested in comparing the performance of the dynamic programming (DP) algorithm (Sect. 3) with the greedy heuristic (Sect. 5.1) for tree topologies and when used for alleviating congestion in general network topologies. The metrics used to measure the performance of the heuristics are the improvement in the network congestion and the cost of the new links needed for network augmentation. We measure these metrics for different network topologies and as a function of the bandwidths of the links available for augmentation. For the SPRT of a network we are interested in the minimum cost of the links needed to alleviate its congestion, as a function of the bandwidths of the links available for augmentation.

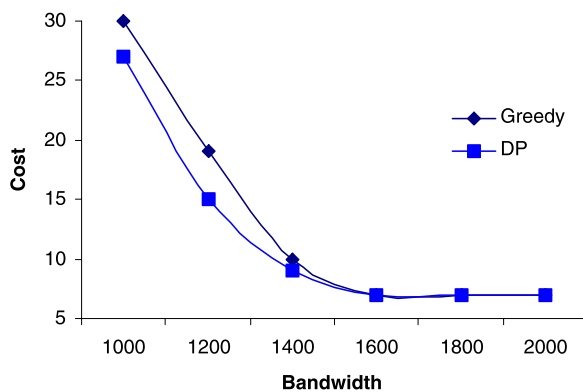
The data for our simulations comprises five independent management domains in an actual service provider's ATM network. We call these domains A, B, C, D, E . Each domain consists of one management gateway (MGR) which forms the root of the SPRT for that domain. The management traffic between the switches and the MGR is routed over the unique paths in the SPRT of the domain. The domains A, B, C, D, E have 57, 55, 67, 76, 41 nodes (switches) and 143, 143, 138, 163, 94 links, respectively. Some of the links in these 5 domains have 100% of their bandwidth (about 1.5 Mbps) designated to carry only management traffic. Other links with bandwidth ranging from 40 Mbps to 599 Mbps have a fixed proportion of their bandwidth designated for management traffic (0–5%). A link (in the SPRT) is considered congested for management traffic if the amount of management traffic routed on the link exceeds the bandwidth designated on the link for management traffic. This congestion is defined by the maximum ratio over all links of the management traffic load of a link to the bandwidth of the link designated for management traffic. The management requirement of the switches in the domains varies from 50 Kbps to 2 Mbps. We measured the amount of congestion in the five domains before augmentation at 96, 5, 218, 414, 91 respectively. Since these domains belong to an actual service provider's network we are unable to provide any details on their topologies and related attributes.

Our first set of results are for the case in which all the links used for augmentation have the same bandwidth. Thus in this scenario a congestion free routing tree is computed by adding new links of the same bandwidth to the SPRT. Here we are interested in doing sensitivity analysis of our heuristics for varying values of this common bandwidth. These results are shown in Fig. 2 and Fig. 3, as a function of the common bandwidth of the links used for augmentation (on the X axis).

For the case in which links used for augmentation have the same bandwidth we evaluated the performance of both the DP and Greedy tree algorithms in terms of the number of new links added (Cost on the Y -axis) to the SPRT. We only show the full results (on all domains) for the DP and provide a comparison between the DP and the Greedy heuristic on one domain where they exhibit the most difference. On all other domains their performance is almost identical. The results for the DP are shown in



(a) Dynamic Programming (DP) based Heuristic

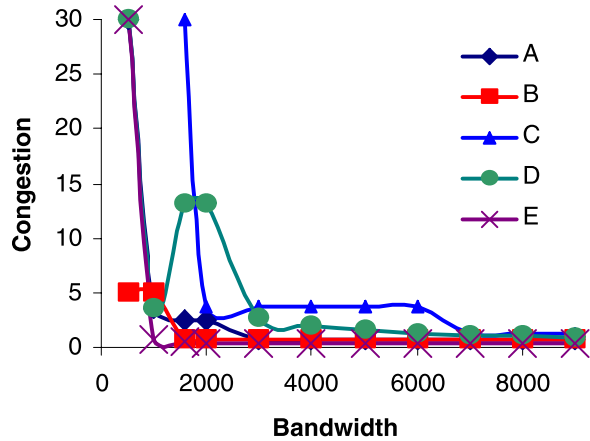


(b) DP vs Greedy Algorithm based Heuristic on Domain D

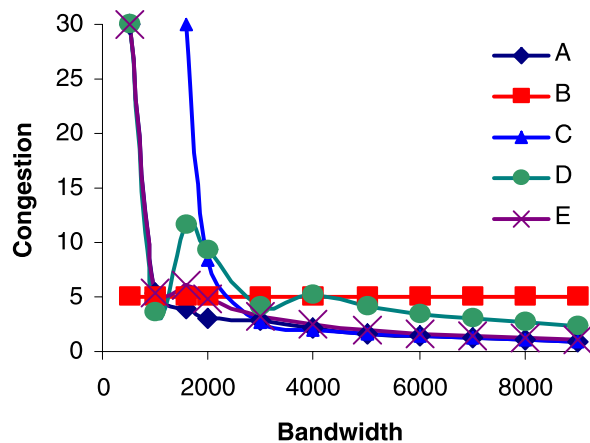
Fig. 2 Cost improvement with increasing (uniform) augmentation link bandwidth

Fig. 2a. Here the common bandwidth of the links used for augmentation is plotted on the X axis and the minimum number of links needed for augmentation (cost) is on the Y axis. As shown in this figure, starting at a link bandwidth of approximately 1000 Kbps, a congestion-free rooted tree is possible in some domains. At approximately 2000 Kbps, congestion-free rooted trees are possible in all domains. Note also that except for one domain, where 7 new links are needed, all domains can be made to have a congestion-free rooted tree by adding at most 3 links. Note that even though the DP can always find the minimum number of links required it may not be the case for the Greedy algorithm. This is exhibited in Fig 2b which compares the performance of the Greedy algorithm with that of the DP algorithm for domain D , in terms of the number of new links added to the SPRT. Here also the minimum number of links needed for augmentation is on the Y axis which is plotted as a function of the common bandwidth of the links used for augmentation (X axis).

Fig. 3 Congestion improvement with increasing (uniform) augmentation links bandwidth



(a) Dynamic Programming (DP) based Heuristic

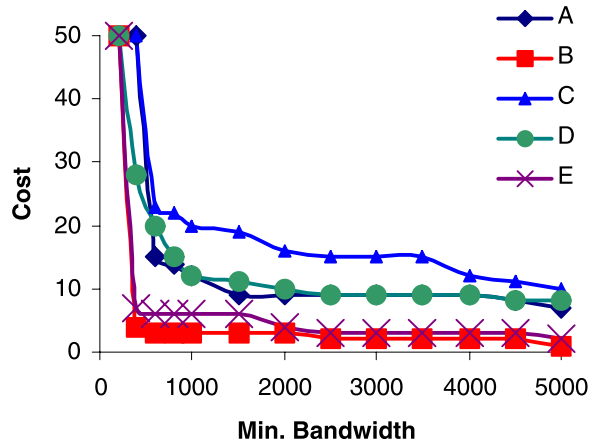


(b) Greedy Algorithm based Heuristic

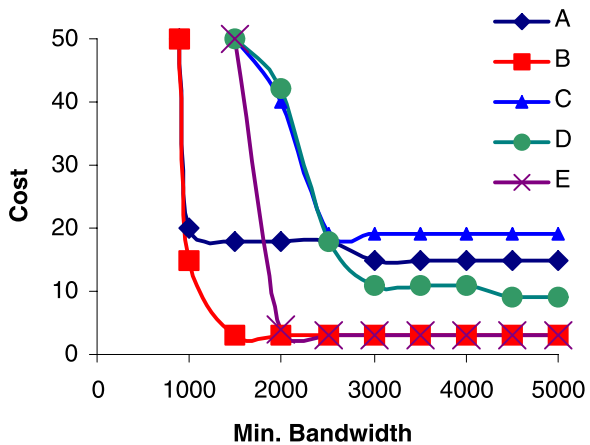
Next, for the case in which links used for augmentation have the same bandwidth, we evaluated the performance of the heuristic for general network topologies (Algorithm 5). We obtain two sets of results based on whether the underlying tree algorithm is DP or the Greedy heuristic. The new congestion (depicted on the Y axis) after applying Algorithm 5 is shown in Fig. 3 for both the Greedy and DP based heuristics. This is plotted as a function of the common bandwidth of the links used for augmentation (X axis). Both the Greedy and DP based heuristics exhibit somewhat erratic behavior at small bandwidths (between 1000–3000 Kbps), which may be attributed to the possibility that at small bandwidths these new links are prone to congestion since they are close to the root. However, at higher bandwidths (7000 Kbps and above) the DP based heuristic is able to alleviate congestion in all the domains.

The next set of results are for the case in which the bandwidth and the cost of the links used for augmentation varies with the node on which they are incident, with higher bandwidth links costing more. The bandwidth of these links vary uniformly

Fig. 4 Total cost of new links added to the SPRT for increasing (non-uniform) minimum augmentation links bandwidth



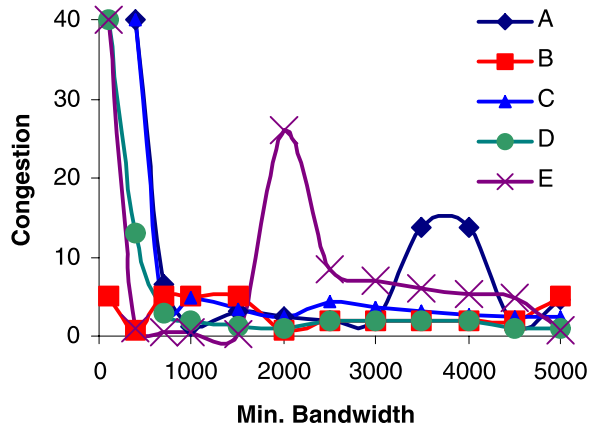
(a) Dynamic Programming (DP) based Heuristic



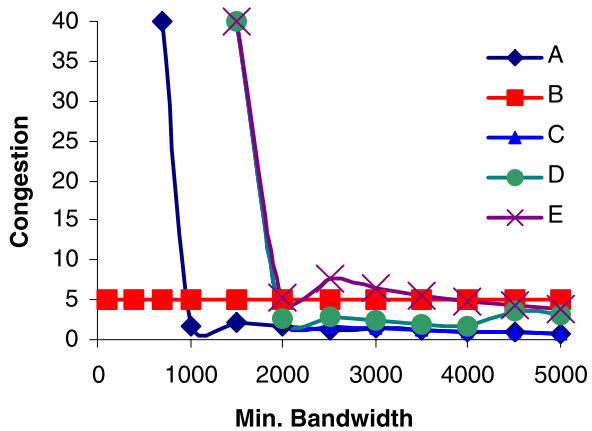
(b) Greedy Algorithm based Heuristic

within a factor 16 of the least bandwidth value, and the link costs vary from 1 to 4, with higher bandwidth links costing more. Here we plot the minimum bandwidth of the links used for augmentation on the X axis. Figure 4 depicts on the Y axis the minimum total cost of the augmentation links needed for guaranteeing congestion-free rooted trees for both the DP and the Greedy algorithms. Note that these results show that the Greedy algorithm may not even find a feasible solution in some cases, even though there exists one which the DP algorithm is guaranteed to find. Figure 5 shows the performance of the heuristic defined in Sect. 5.2 (Algorithm 5), in terms of its ability to alleviate congestion in the new network, both when the underlying tree augmentation is based on the DP and the Greedy algorithms. Here the congestion value for the links of the augmented network (the new SPRT) is shown on the Y axis which is plotted as a function of the minimum bandwidth of the links used for augmentation (X axis). Note that in this case the DP based heuristic shows somewhat erratic behavior. A possible reason is that a solution of minimum cost, which the

Fig. 5 Congestion improvement with increasing (non-uniform) minimum augmentation links bandwidth



(a) Dynamic Programming (DP) based Heuristic



(b) Greedy Algorithm based Heuristic

DP is guaranteed to find, may use a large number of links each of small cost. With more links modified in the original network, the variability in the resulting congestion increases. However, the Greedy algorithm does not take into account the variability in the cost of the links and hence attempts to minimize the total number of modified links, thus exhibiting less erratic behavior.

Our results show that in almost all cases the congestion in management flows can be brought down significantly by using the heuristic defined in Sect. 5.2. In addition, only a small number of augmenting links of low bandwidth are needed for this purpose. It also appears that at lower bandwidths (of the links used for augmentation), the DP algorithm based heuristic is able to find a good solution even when the Greedy algorithm based heuristic does not succeed in alleviating congestion.

6 Conclusion and Open Problems

In this paper we designed efficient heuristics for traffic engineering of management traffic in data networks. We showed both analytically and by simulation that these heuristics have good performance in service provider networks. Our work raises several open questions. For the budget-constrained network augmentation problem, it is still open to find an algorithm that would satisfy all or none of the demand at each vertex, i.e., when the demand at a vertex is not splittable. It is desirable to develop a PTAS for this problem. However, it is not clear how to do this with hard budget and capacity constraints. We conjecture that our algorithm solves this problem with a violation of at most a factor of $1 + \epsilon$ of the capacity constraints.

References

1. Applegate, D., Cohen, E.: Making intra-domain routing robust to changing and uncertain traffic demands: understanding fundamental tradeoffs. In: ACM SIGCOMM, 2003
2. Bejerano, Y., Rastogi, R.: Robust monitoring of link delays and faults in IP networks. In: IEEE INFOCOM, 2003
3. Bienkowski, M., Korzeniowski, M., Räcke, H.: A practical algorithm for constructing oblivious routing schemes. In: 15th Annual ACM Symp. on Parallel Algorithms and Architectures (SPAA), pp. 24–33 (2003)
4. Breitbart, Y., Chan, C.Y., Garofalakis, M., Rastogi, R., Silberschatz, A.: Efficiently monitoring bandwidth and latency in ip networks. In: IEEE INFOCOM, 2001
5. Chen, J., Rajaraman, R., Sundaram, R.: Meet and merge: approximation algorithms for confluent flow. In: Symp. on Theory of Computing (STOC), pp. 373–382 (2003)
6. Chen, J., Kleinberg, R., Lovasz, L., Rajaraman, R., Sundaram, R., Vetta, A.: (Almost) tight bounds and existence theorems for confluent flows. In: Symp. on Theory of Computing (STOC), pp. 529–538 (2004)
7. Ford, L., Fulkerson, D.: Flows in Network. Princeton University Press, Princeton (1962)
8. Fortz, B., Thorup, M.: Optimizing OSPF/IS-IS weights in a changing world. IEEE J. Sel. Areas Commun. **20**(4), 756–767 (2002)
9. Fortz, B., Rexford, J., Thorup, M.: Internet traffic engineering by optimizing OSPF weights. In: IEEE INFOCOM, pp. 118–124 (2000)
10. Fortz, B., Rexford, J., Thorup, M.: Traffic engineering with traditional IP routing protocols. IEEE Commun. Mag. **40**(10), 118–124 (2002)
11. Frank, A.: Connectivity augmentation problems in network design. In: Mathematical Programming: State of the Art, pp. 34–63 (1994)
12. Iyer, S., Bhattacharaya, S., Taft, N., Diot, C.: An approach to alleviate link overload as observed on an IP backbone. In: IEEE INFOCOM, 2003
13. Jamin, S., Jin, C., Jin, Y., Raz, D., Shavitt, Y., Zhang, L.: On the placement of Internet instrumentation. In: IEEE INFOCOM, 2000
14. Kleinberg, J.: Approximation algorithms for disjoint paths problems. Ph.D. Thesis, MIT (1996)
15. Kumar, A., Rastogi, R., Silberschatz, A., Yener, B.: Algorithms for provisioning virtual private networks in the hose model. In: ACM SIGCOMM, pp. 135–148 (2001)
16. Li, L., Thottan, M., Yao, B., Paul, S.: Distributed network monitoring with bounded link utilization in ip networks. In: IEEE INFOCOM, 2003
17. Lorenz, D., Orda, A., Raz, D., Shavit, Y.: How good can IP routing be? In: DIMACS Technical Report: 2001-17 (2001)
18. Wang, J., Nahrstedt, K.: Hop-by-hop routing algorithms for premium-class traffic in diffserv networks. In: IEEE INFOCOM, 2002